Workshop 05 Overview

This Python script **performs crime clustering analysis** using **Hierarchical Clustering** on crime data from the **West Midlands, UK** and visualises the results on a **map using Folium**. The main tasks performed in the script include:

Loading the Crime Dataset – Reads crime data from a CSV file and drops unnecessary columns.
Data Preprocessing & Cleaning – Handles missing values and extracts necessary features.
Data Transformation – Standardizes the dataset for clustering.
Hierarchical Clustering – Groups crime locations into clusters using the **Ward method**.
Dendrogram Analysis – Determines the optimal number of clusters.
Assigning Crime Clusters – Labels crime incidents with cluster numbers.
Visualising Clusters on a Map – Displays crime clusters on an interactive **Folium map**.
Saving Results – Saves clustered data and the crime map for future use.

Step-by-Step Breakdown

Importing Required Libraries

Pandas – Handles tabular data processing.
NumPy – Supports numerical computations.
Matplotlib & Seaborn – Creates dendrograms and visualizations.
Scikit-learn (StandardScaler) – Standardizes data for clustering.
SciPy (Hierarchical Clustering) – Performs clustering with linkage & dendrograms.
Folium – Creates an interactive map for crime clusters.

Loading the Crime Dataset

- **Loads the crime dataset** from a CSV file.
- **Removes unnecessary columns** (Crime ID and Context) to simplify analysis.

Data Preprocessing & Cleaning

- **Displays dataset information,** including column names and data types.
- **Removes rows missing latitude or longitude**, as these are necessary for mapping.

Data Transformation for Clustering

- **Extract only location features (Latitude & Longitude)**.
- **Applies Standardization** to normalize the scale of the data.

Applying Hierarchical Clustering

- **Performs Hierarchical Clustering** using the **Ward method** (minimises variance within clusters).
- **Creates a Dendrogram** to analyse cluster formations.

Assigning Crime Clusters

- **Assign each crime incident to a cluster** using a **cut-off distance**.
- **Checks the number of unique clusters** and their sizes.
- **Ensures the clustering process is successful**.

Visualising Crime Clusters on a Map

- **Initializes a Folium Map** centered around Birmingham.
- **Assign unique colours to different clusters**.
- **Adds crime locations to the map with cluster-based colouring**.
- **Saves the map to an HTML file**.
- **Opens the saved map in the web browser**.

Saving Clustered Data

- **Saves the processed dataset with cluster labels**.

List of Main Functions Used

Function	Purpose
pd.read_csv()	Loads the dataset from a CSV file
df.drop()	Removes unnecessary columns
df.dropna()	Drops rows with missing latitude/longitude
StandardScaler().fit_transform()	Standardizes data for clustering
linkage()	Performs hierarchical clustering
dendrogram()	Creates a dendrogram for analysis
fcluster()	Assigns clusters to data points
folium.Map()	Initializes an interactive map
folium.CircleMarker()	Adds crime locations to the map
MarkerCluster().add_to()	Groups markers for better visualisation
df.to_csv()	Saves the clustered data to a CSV file
webbrowser.open()	Opens the saved HTML map in a browser

Summary of Key Insights

✓ Crime incidents were clustered using Hierarchical Clustering.
✓ Clusters were assigned based on geographical proximity.
✓ A dendrogram was used to determine the optimal clustering distance.
✓ An interactive Folium map was generated to visualise crime hotspots.
✓ The clustered dataset was saved for further analysis.